Testing, Packaging, and Tooling

1. Testing with pytest

`pytest` is the de facto standard testing framework for Python, offering concise assertions, powerful fixtures, parameterization, and extensive plugin ecosystems. Test functions must start with `test_`.

Fixtures manage test setup and teardown cleanly using dependency injection. Parameterization allows running the same test function across multiple datasets effortlessly.

```
import pytest @pytest.fixture def sample_data(): return [1, 2, 3] @pytest.mark.parametrize('input,expected', [(1, 2), (2, 4)]) def test_double(input, expected): assert input * 2 == expected
```

2. Mocking and Stubbing

The `unittest.mock` library enables replacing parts of your system under test and making assertions about how they were used. The `@patch` decorator or context manager replaces target objects during test execution.

Developers should avoid over-mocking (mocking internal implementation details rather than boundary interfaces), which leads to brittle test suites that break during refactoring.

```
from unittest.mock import patch @patch('requests.get') def test_api_call(mock_get): mock_get.return_value.status_code = 200 assert mock_get.return_value.status_code == 200
```

3. Modern Packaging with pyproject.toml

PEP 518 and PEP 621 standardized Python project metadata and build system requirements into a single `pyproject.toml` configuration file, replacing legacy `setup.py` and `setup.cfg` files.

Packages are built using standard PEP 517 frontends such as `python -m build`, generating source distributions and wheel files ready for PyPI distribution.

```
[build-system] requires = ['setuptools>=61.0.0'] build-backend =
'setuptools.build_meta' [project] name = 'my_package' version = '0.1.0'
```

4. Linting, Formatting, and Static Analysis

Maintaining high code quality requires automated linters and formatters. Modern Python tooling has consolidated around extremely fast Rust-based tools like Ruff (replacing Flake8, isort, and pylint), Black (opinionated formatting), and Mypy (static type checking).

Pre-commit hooks automate running these checks locally before every git commit, ensuring non-compliant code never reaches the remote repository.

```
# Run Ruff check and Black formatting ruff check . black . --check
```

5. Continuous Integration (CI)

Continuous Integration pipelines, such as GitHub Actions, automate building, testing, and linting across multiple operating systems and Python versions (matrix builds). Test coverage reports are routinely uploaded to services like Codecov to monitor code quality metrics.

```
name: CI on: [push, pull_request] jobs: test: runs-on: ubuntu-latest steps: - uses:
actions/checkout@v4 - name: Set up Python uses: actions/setup-python@v5 with:
python-version: '3.12'
```